

SQL SERVER

Muhammad Anwaar Ur Rehman

LESSON: 1

MS SQL INSTALLATION

LESSON: 2

SQL MANAGEMENT STUDIO

LESSON: 3

Create Database in MS SQL Server (Two types)

1. System Data Bases

Master DB

Model DB

MSDB

Tempdb

2. USER Defined Data Bases/ New Query

```
CREAT DATABASE SAMPLE-DB;  
USE SAMPLE-DB;
```

System Databases

Resource



It is the physical repository used to store the system objects that appear within the sys schema of every database.

MSDB



MSDB is used as a metadata repository for many SQL Server features, including Server Agent, Backup/Restore, Database Mail, Log Shipping, Policies, and more.

Master



Master is the system database that contains metadata for instance-level objects, such as logins, linked servers and certificates for implementing encryption.

Model



Model is used as a template for all new databases that are created on the instance.

TempDB

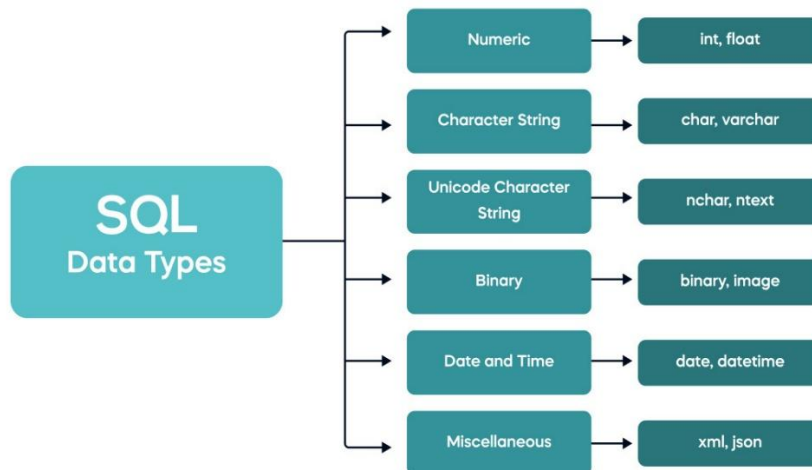


TempDB is a workspace used by SQL Server when it needs to create temporary objects. This applies to temporary tables created by users, and less commonly known, it also applies to table variables.

LESSON: 4

SQL Data Types

- i. **Numeric**
Integer, Decimal, Numeric, Smallint, Bigint, Tinyint, Real, Float, Money, Small money
- ii. **CHARACHAR**
Char, Varchar, Nchar, Nvarchar
- iii. **Temporal**
Date, Time, Datetime, smallDateTime, DateTimeOffset
- iv. **Miscellaneous**
Bit, SQL-Variant, Unique identifier, XML, TABLE, Binary Datatypes, LargeobjectDataTypes



LESSON: 5

Integrity Constraints

- A set of Rules
 - Used to maintain the quality of information
 - Ensure that the data Insertion, updating, and other processes have to be Performed a way that data Integrity m such a way in not affected
- (UNIQUE, NOT NULL, PRIMARY Key, Foreign Key, Default, Check)



LESSON: 6

SQL Statement: INTO

DDL: Data Definition Language

Used to create, modify the objects structure (database, tables, views, triggers)
[CREATE, ALTER, DROP, TRUNCATE, RENAME]

DML: Data manipulation Language

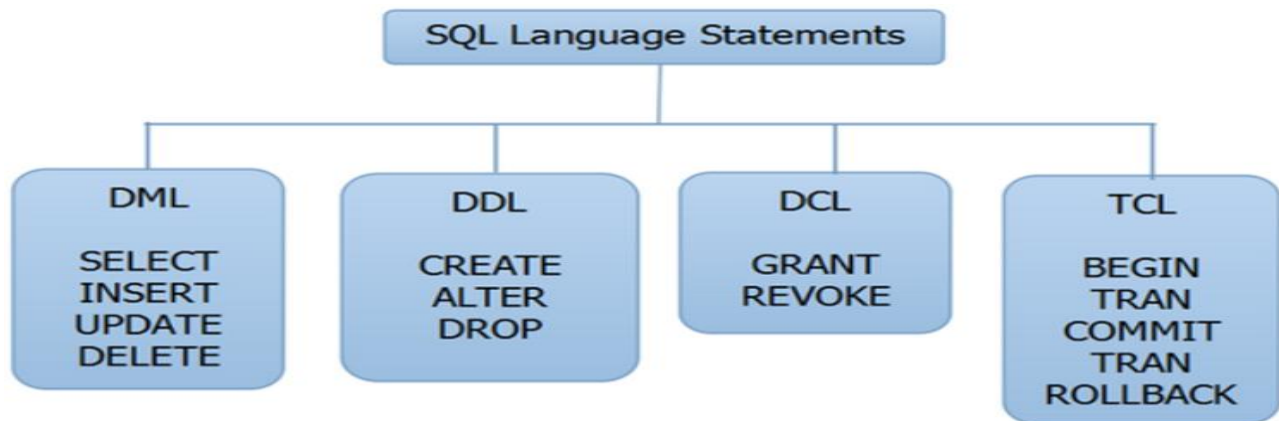
Used to Insert data, modify the existing data, removing & retrieving data from the tables.
[INSERT, DELETE, UPDATE, SELECT]

TCL: Transection Control Language

Used to save data, undo/redo transection performed on the data base.
[COMMIT, Rollback, Save Transection]

DCL: Data Control Language

Used to create roles, permission, referential Integrity and as well it is used to database Control access to securing it. [GRANT, REVOKE]



DDL

LESSON: 7

CREATE STATEMENT

- Used to create object's strictures (database, tables, visas triggers)

Syntax:

```
CREATE DATABASE database_name;  
CREATE DATABASE employeeDB;
```

Practical:

```
CREATE DATDBASE employee DB;  
USE employeeDB;  
CREATE TABLE employee-info(  
EmpID INTEGER KEY,  
EmpName varchar (80) Not Null,  
Empsalary Decimal (10, 2) Not Null,  
Job Varchar (50),  
Phone varchar (80),  
DeptID integer);
```

DML

LESSON: 8

INSERT STATEMENT

- Used to insert data or Information into the table.

Syntax:

```
INSERT INTO table_Name  
values (011-Value, Cola-Value);  
INSERT INTO employee_Info  
Values (01, 'Adam', 20000,  
'Developer', 9879, 10);
```

Practical:

```
INSERT INTO employee_info Values (01, 'ADAM', 25000,  
'Developer', '987997797', 19)  
INSERT INTO employee_info (EmpID, EmpName, Empsalary)  
Values (02, 'Smith', 38000);
```

DML

LESSON: 9

Update Statement

- Used to update/modify existing table data/information

Syntax:

```
UPDATE table_name  
SET Column_name = Column_Value...  
[WHERE Condition];
```

Example:

```
UPDATE employee_info SET EmpSalary =  
EmpSalary+ 1000;  
UPDATE employee_info SET job= 'Tester'  
Where employee camp Name = 'Simits';  
(Specific)
```

Practical:

```
UPDATE employee-Info  
SET EmpSalary = EmpSalary + 1000;  
SELECT * From employee-Info;  
UPDATE employee-Info  
SET Job = 'Tester'  
WHERE EmpName = 'smith'
```

DML

LESSON: 10

Delete Statement

Used to delete one or more recorded from table.

Syntax:

```
DELETE From table-name [WHERE Condition];
```

Example:

```
DELETE FROM employee_info;  
DELETE FROM employee-info WHERE  
deptID=10;
```

Practical:

```
SELECT * From employee-info;  
DELETE FROM employee_info  
where deptID = 10; (only one row can deleted)  
Delete FROM employee_info; (all record deleted)
```

CLAUSE

LESSON: 11

ORDER BY CLAUSE

Used to sort/arrange records in either ascending or descending order. Always used with SELECT Statement

Syntax:

```
SELECT Column-name(s) From table-N  
ORDER BY Column-name [DESC];
```

Example:

```
SELECT * FROM employee-info  
ORDER BY EmpSalary;  
SELECT * FROM employee-info  
ORDER BY employee DESC;
```

Practical:

```
SELECT * FROM employee_info  
ORDER BY EmpSalary DESC;  
SELECT empName FROM employee_info  
ORDER BY EmpName DESC;
```

CLAUSE

LESSON: 12

WHERE CLAUSE

Using WHERE clause, you are able to restrict query to rows that meet a Condition. You can use any operation in WHERE Clause. (SELECT, UPDATE, DELETE) DML. Only Insert Statement Not in used

Syntax:

```
SELECT * FROM table name  
WHARE Condition;  
UPDATE table name SET  
Column-name=value  
WHERE Condition;  
DELETE FROM table_name  
WHERE Condition;
```

Practical:

```
SELECT * FROM employee-info;  
SELECT EmpName, EmpSalary FROM employee_info  
SELECT * FROM employee.info  
WHERE deptID= 10; [Developer, 11, 11]  
UPDATE employee-info SET EmpSalary= EmpSalary+5000  
WHERE deptID=20;  
DELETE FROM employee_info  
WHERE deptID = 30;
```

FUNCTION

LESSON: 13

AGGREGATE FUNCTION

SUM (), AUG (), MIN (), MAX (), COUNTO

Practical:

```
USE employeeDB;  
SELECT * FROM employee-info;  
SELECT SUM(EmpSalary) FROM eemploy.info;  
SELECT * FROM employee_info  
INSERT INTO employee_info (EmpID, EmpName, EmpSalary)  
Values (6, 'POB', 15000);
```

DML

LESSON: 14

NUMERIC FUNCTION

SELECT ABS (-1); / SELECT CEILING (76.5); / SELECT FLOOR/76.12); / SELECT SIGN (15) Save
SQUAREC); / SQRT (81); / PI (); / cos (); /sin (); /tan (); / EXP ();

DML

LESSON: 15

STRING FUNCTION

Practical:

```
SELECT LEN (DATABASE)
SELECT EmpName, LEN(EmpName) FROM employee_info;
SELECT UPPER ("Capitelled")
SELECT LOWER (".....")
SELECT LTRIM ("...."); RTRIM ("....")
SELECT SUBSTRING ('MICROSOFT', 16, 9)
SELECT REPLACE('MICROSOFT', 'MKRO'; "MIJOR");
SELECT Replicate("..." ,5);
```

OPERATOR

LESSON: 16

Boolean Operator

Practical:

```
CREATE TABLE tbl_user_login_info(
User_id INT PRIMARY KEY IDENTITY;
Username VARCHAR(50) NOT NULL UNIQUE,
Email VARCHAR(50) NOT NULL UNIQUE,
Password VARCHAR(50) NOT NULL);
INSERT INTO tbl_user_login_info(Username, Email Password)
VALUES ('Ironman', 'tony@stark.com' -'tony123'), ('captain', 'Steve@captain.com', 'Steve345'), ('spider',
'Peter@parker.com', 'Peter123'), ('wonder', 'wonder @woman.com', 'wonder123');
SELECT * FROM tbl_user_Login_info;
SELECT username FROM tbl_user_login_info
WHERE Email = 'Steve@captain.com';
AND Password = 'Stene@456'; (All are correct)
SELECT Password FROM tbl_user_login_info
WHERE username = 'spider' OR email = 'peter@parker.com'; (any one correct)
WHERE NOT UserName = 'Wonder; (! <, >, !=)
```

FUNCTION

LESSON: 17

DATE-TIME FUNCTION

- GET DATEU, Get Date(), Current_timestamp, sysdatetime(), Sysdatetime()
- date name(), datediff(), Age, dateadd()

Practical:

```
SELECT GETDATE(); / Sysdatetime( ); / Current-Times tamp;  
SELECT DATENAME (MONTH, CURRENT- Timestamp)  
AS 'MONTH'; / AS 'YEAR';  
SELECT DATEDIFF('Year', '24 Aug 2002, Current_timestamp) AS "Ag";  
SELECT DATEADD('YEAR', 100, Current_timestamp) AS "100 years From now";
```

CLOUSE

LESSON: 18

Group By CLAUSE

- Defines one of move columns as a group such the all rows within any gray have the Same Values for those columns.
- Always used with. SELECT statement

Syntax:

```
SELECT Column-name(s), aggregate_function ()  
FROM table-name Group BY Column_name;
```

Example:

```
SELECT deptID FROM employee_info Group By  
deptID;  
SELECT deptID, sum(EmpSalary) FROM  
empl0yee_info Group By deptID;
```

Practical:

```
SELECT * FROM employee_info;  
SERECT DEPTNO FROM Employee_info  
Group BY DeptNo;  
SELECT SUM (Salary) ) FROM  
empl0yee_info Group By deptID;  
SELECT DEPINO Sum (Salary) AS 'Total  
Salary' FROM Employee_info Group BY  
DEPTNO;
```

CLAUSE

LESSON: 19

HAVING CLAUSE

The having clause defines the condition that is then applied to Groups of rows. Always used with Select statement inside Group by Clause

Syntax:

```
SELECT Column_names(s), aggregate function()  
FROM table_name Group Of Column_name  
HAVING Condition;
```

Example:

```
SELECT deptID Sum(EmpSalary)  
FROM employee_info  
Group By deptID Having deptID = 20;
```

Practical:

```
SELECT * FROM employee_info;  
SELECT DEPTNO, SUM(Salary) AS 'Total Salary'  
FROM employee_info  
Group By DEPTINO Having DEPTNO = 20;  
SELECT DEPTNO, SUM(Salary) AS 'Total Salary'  
FROM employee_info  
WHERE DEPTINO = 20 Group By DEPTINO;
```

CLAUSE

LESSON: 20

TOP CLAUSE

- Top Clause specifies the first N rows of the query that are to be retrieved.
- This Clause should always be used with the ORDER BY clause.

Syntax:

SELECT TOP(n) Column_name FROM table-
Name ORDER By Column_name [DESC];

Example:

SELECT TOP(3) EmpSalary FROM
employee_info ORDER BY EmpSalary DESC;

Practical:

SELECT * FROM employee_Info;
SELECT Salary from employee_info
ORDER By Salary desc;
SELECT TOP (3) Salary FROM employee_info
ORDER BY salary;

LESSON: 21

Copy Data From one DB to Another

- Create a copy of the table from different Database

Syntax:

SELECT Column_Name/* INTO table_name
FROM database_name, table_name;
SELECT * INTO employee_details
FROM employee_db, employee_info;

Example:

FROM EMP_DB. DBO. EMP_INFO;
SELECT * From EMP-DETAILS;

Practical:

USE EMP-DB;
SELECT * From EMP_INFO;
CREATE DATABASE SAMPLE_EM_DB;
USE SAMPLE_EM_DB;
SELECT * INTO EMP_DETAILS

DDL

LESSON: 22

ALTER STATEMENT (Part_1)

ALTER TABLE

- Modifies a table definition by adding, altering, or dropping Columns and Constraints,
- It also Reassigns and rebuilds Partitions, or disables and enables Constraints and triggers.

Adding a New Column

- Add a column without Constraint that allow Null values.

Syntax:

ALTER TABLE table_name ADD
Column_Name datatype (size) [Null];

Example:

ALTER TABLE Emp_info ADD Salary
decimal;

Practical:

USE EMP-DB;
CREATE TABLE Emp-details (ID int, name
varchar(10));
Select * from Emp_details;
ALTER TABLE Emp_details ADD Salary decimal;

```
ALTER TABLE Emp_info ADD phone  
varchar(10) NULL;
```

```
INSERT into Emp_details values(1, 'Anwaar', 2000);  
ALTER TABLE Emp_details ADD phone voucher(10)  
NULL;
```

Adding Column with a Constraint

- Adds a new column with constraint (unique, Default, etc.)

Syntax:

```
ALTER TABLE table_name ADD Column_name  
datatype(size) Constraint [null];  
ALTER table_name App Column_Name datatype(s)  
NULL
```

```
Constraint Constraint_reference_name constraint;
```

Example:

```
ALTER TABLE Emp_info ADD Scary decimal not NULL  
ALTER TABLE Emp_info ADD Project_ID integer NULL  
Constraint PID_unique_Key unique;
```

Practical:

```
ALTER TABLE Emp_details ADD  
Project_Completed int not null Default 5;  
ALTER TABLE Emp_details ADD Project_ID  
integer NULL
```

```
Constraint PID_unique_Key Unique;
```

```
select * From Emp_details;
```

```
insert into emp-details values (2, 'Stave',  
5000, '1078978871', default, 4);
```

Adding several columns with Constraints

- Add more than one columns with Constraints defined with the New Columns.

Syntax:

```
ALTER TABLE table_Name  
ADD Column_Name datatype (size) Constraint,  
ALTER TABLE Emp_info  
ADD salary decimal DEFAULT 15000, Project_ID integer  
null CONSTRAINT  
PID_unique_Key UNIQUE;
```

Practical:

```
CREATE table Company_info (id int);  
ALTER TABLE Company_info  
ADD revenue decimal (10,2) null, Project_ID  
integer Constraint Project_ID PRIMARY  
KEY;  
select * from company-info;
```

DDL

LESSON: 23

ALTER STATEMENT (Part-2)

Dropping a Columns or Columns

- Remove a Column of multiple columns

Syntax:

```
ALTER TABLE table_name DROP COLUMN  
Columns(s);
```

Example:

```
ALTER TABLE Emp_info DROP COLUMN Salary;  
ALTER TABLE Emp_info DROP COLUMN Salary,  
age;
```

Practical:

```
SELECT * INTO EMP_DETATES  
From EMP_DB.DBO.EMP_DETAILS;  
Select * From Emp-info;  
ALTER TABLE EMP-INFO DROP COLUMN  
DEPT_ID;  
DROP TABLE Emp_info;
```

Dropping Constraints & Columns

- Removes a Constraint
- Removes constraints & Columns

Syntax:

```
ALTER TABLE table_name DROP Constraint  
Constraint_name;  
ALTER TABLE table_name DROP Constraint  
Constraint_name, columns Column_Name;
```

Example:

```
ALTER TABLE Emp_info DROP CONSTRAINT  
PID_unique_Key;  
ALTER TABLE Emp_info DROP CONSTRAINT  
PID_unique_Key COLUMN Salary;
```

Practical:

```
SELECT * FROM EMP_INFO;  
DROP TABLE EMP_INFO;  
SELECT * FROM EMP_DETAILS;  
ALTER TABLE EMP_DETAILS DROP  
CONSTRAINT PID_unique_Key;  
SELECT * FROM Company_info;  
ALTER TABLE COMPANY_INFO DROP  
CONSTRAINT Project_PRIMARY KEY;  
COLUMN revenue, Project_ID;
```

DDL

LESSON: 24

ALTER STATEMENT (Part-3)

Changing the data type of a Column

- Changes a column of a table from one data type to another

Syntax:

```
ALTER TABLE table-name ALTER Column Column_name datatype (s);
```

Example:

```
AFFALTER TABLE Emp_info ALTER Column salary decimal (8,2);  
ALTER TABLE Emp-info ALTER Column Emp_name varchar(50);
```

Practical:

```
USE EMP-DB;  
Create Table Customer (ACCNO, CUST_Name CHAR(100));  
ALTER TABLE Customer ALTER Column CUST-Name VARCHAR(100);
```

Changing the Size of a Column

- Change (increase of Decrease) the size of a Column.

Syntax:

```
ALTER TABLE table_name ALTER Column Column_name  
datatype (size);
```

Example:

```
ALTER TABLE emp-info ALTER Column Salary decimal (8,2);  
ALTER TABLE Emp_info ALTER Column Emp_name  
varchar(50);
```

***Note:** If The Columns Contains data, The Column size Can only be increased

Practical:

```
SELECT * FROM CUSTOMER;  
INSERT INTO CUSTOMER  
VALUES (12456, 'Anwaar khan')  
ALTER TABLE CUSTOMER  
ALTER COLUMN  
CUST_NAME VARCHAR(15);
```

Changing datatype and size At A time

- Change the data type as well as the size of a column at some size

Syntax:

ALTER TABLE table_Name ALTER COLUMN Column_Name datatype (s);

Example:

ALTER TABLE Emp-info ALTER COLUMN Salary decimals(8, 2);

ALTER TABLE Emp_info ALTER COLUMN Emp_name varchar(50);

Practical:

SELECT * FROM CUSTOMER;

ALTER TABLE CUSTOMER ALTER COLUMN CUST_NAME VARCHAR(50);

ALTER TABLE CUSTOMER ALTER COLUMN CUST_NAME CHAR(50);

LESSON: 25

Aliases

- Can be used to create a temporary name for Columns or tables.

Types

- 1) **Column Aliases:** Are used to make Column headings in query output easier to read. (Functions & Column Contatenation)
- 2) **Table Aliases:** Are used to shorten your SQL to make it easier to read (join, & Subquery)

Column Aliases

Syntax:

SELECT Column_Name[Function()] AS
alias_name FROM table_name;
SELECT Column_name [Function()] alias_name
FROM table_name;

Example:

SELECT Sal As Salary FROM emp;
SELECT f-name|| name "Full Name" From emp;

Practical:

USE COMPANY_DB;
SELECT * FROM Employee;
SELECT Emp_Salary FROM employee;
SELECT Emp_Salary AS 'Employee_Salary'
FROM employee;

LESSON: 26

Joins

- Used to Retrieve Data from multiple tables.

Types

- 1) Inner join
- 2) outer join
 - i. Lift outer join
 - ii. Right outer join
 - iii. Full outer join
- 3) Cross Join

***NOTE:** At Least one Column have same information in Bath Table

JOIN

LESSON: 27

Inner Join

The inner join is one of the most commonly used joined in SQL Server. It Return all rows from multiple tables where the Join Condition is Satisfied.

Syntax:

```
SELECT Column name(s) FROM table_name INNER join  
table_name  
ON table_name Column_Name = table  
2_name.Column_name
```

Example:

```
SELECT Emp_ID, Emp_name, Emp_Salary, dept_name,  
dept_location From employee INNER Join department  
ON employee. Emp_dept_id=department. Dept_id;  
SELECT e.emp_id, e.emp_name, e.emp_Salary,  
d.dept_name, d.dept_location FROM employee INNER  
JOIN department.d  
ON e.emp_deptid = d.dept_id;
```

Practical:

```
USE COMPANY_DB;  
SELECT * FROM employee;  
SELECT * FROM department;  
SELECT Emp_ID, Emp_name,  
Emp_Salary, dept_name, dept_location  
From employee INNER Join department  
ON employee. Emp_dept_id=department.  
Dept_id;  
SELECT e.emp_id, e.emp_name,  
e.emp_Salary, d.dept_name,  
d.dept_location FROM employee INNER  
JOIN department.d  
ON e.emp_dept_id = d.dept_id;
```

JOIN

LESSON: 28

Left Outer Join

Return all rows from the left-hand table and records in the right-hand table with matching values.

Syntax:

```
SELECT Column_Name(s)  
FROM table1_name LEFT OUTER JOIN table2_name  
ON table1_name.Column_name=table2_name.Column_name;
```

***Note: The Records without matching values are replaced with Nelly in the respective columns.**

Example:

```
SELECT Emp_ID, Emp_name, Emp_Salary, dept_name,  
dept_location  
From employee LEFT OUTER JOIN department  
ON employee.Emp_dept_id = department. Dept_id;  
SELECT e.emp_id, e.emp_name, e.emp_Salary, d-dept_name,  
d.dept_location  
FROM employee e LEFT OUTER JOIN department d  
ON e.emp_dept_id = d.dept_id;
```

Practical:

```
USE COMPANY-DB  
SELECT Emp_ID, Emp_name,  
Emp_Salary, dept_name,  
dept_location  
From employee LEFT OUTER  
JOIN department  
ON employee.Emp_dept_id =  
department. Dept_id;  
SELECT e.emp_id, e.emp_name,  
e.emp_Salary, d-dept_name,  
d.dept_location  
FROM employee e LEFT OUTER  
JOIN department d  
ON e.emp_dept_id = d.dept_id;
```

JOIN

LESSON: 29

Right Outer Join

- Return all rows from the right-hand table and the left hand table with matching records Values.

Syntax:

```
SELECT Column_name(s)
FROM table_name Right outer Join table2_Name
ON table_name,
Column_name=table2_name.column_name
```

***Note: The recorded without matching values are replaced with NULLs in the respective Columns**

Example:

```
SELECT Emp_ID, Emp_name, Emp_Salary, dept_name,
dept_location
FROM employee RIGHT OUTER JOIN department
ON employee.Emp_dept_id = department.dept_bd;
```

Practical:

```
USE COMPANY_DB;
SELECT Emp_ID, Emp_name,
Emp_Salary, dept_name, dept_location
FROM employee RIGHT OUTER JOIN
department
ON employee.Emp_dept_id =
department.dept_bd;
```

JOIN

LESSON: 30

Full outer Join

- Return all rows from both left hand and Right hand wilt matching values.

Syntax:

```
SELECT Column_name(s)
FROM table1-name Full OUTER JOIN tables2_Name
ON table1_name.Column_mane = table2_name.columns_name
Note: The Records without matching values are replaced with NULLS in the respective columns.
```

Example:

```
SELECT Emp_ID, Emp_name, Emp_Salary, dept_name,
dept_location
FROM employee FULL OUTER JOIN department
ON employee.Emp_dept_id = departement.dept_id;
```

Practical:

```
USE COMPANY_DB;
SELECT Emp_ID, Emp_name,
Emp_Salary, dept_name,
dept_location
FROM employee FULL OUTER
JOIN department
ON employee.Emp_dept_id =
deparetment.dept_id;
```

LESSON: 31

Subquery

1. Display name, salary of the employee's whose salary is greater Than Mike's Salary?
2. Display Name, Salary of the employees whose is greater than Adams Salary if and dept_NO same as Adams dept_NO?
3. Display the Employee information where department is Located at New York?
1. A query with in another SQL query and embedded within the **WHERE** Clause.
2. Subquery must be enclosed with in parenthesis()
3. Subquery Can be used with the **SELECT**, **INSERT**, **UPDATE**, and **DELETE** statements along with the Comparison operators.
4. A Subquery Can be have only one Column in the **SELECT** Statement.

Syntax:

```
SELECT Column. Name's  
FROM table_Name  
WHERE Column_name OPERATOR  
(SELECT Column_name FROM table_Name  
[WHERE Condition]);
```

Example:-

```
SELECT emp-name, emp-salary  
FROM employee  
WHERE Emp_Salary = (SELECT emp_salary FROM  
employee Where Emp-name='MIKE');  
SELECT Emp_name, Emp_Salary FROM employee  
WHERE Emp_Salary IN (SELECT Emp_Salary FROM  
employee  
WHERE Emp_name = 'MIKE');
```

Practical:

```
SELECT * FROM employee;  
SELECT emp-Name, emp-Salary FROM  
employee  
WHERE Emp_Salary > (SELECT  
Emp_Salary FROM employee  
WHERE Emp_name = 'Mike');  
SELECT emp-name, Emp_Salary  
FROM employee WHERE Emp_Salary >  
(SELECT Emp_Salary FROM employee  
WHERE emp name 'Adam') And  
emp_dept_id = (SELECT emp_dept_id  
FROM employee WHERE Emp_name =  
'Adam');
```

T-SQL

LESSON: 32

Transact-SQL (T-SQL)

1. Transact SQL is better Known as T_SQL
2. The purpose of T-SQL is used to provide a Set of tools for the development of a transactional Databases.

Standard SQL

DML Statements
Operators
Built-IN Functions
Single-line Query

T-SQL

Standard SQL
Batch or script
Triggers
Work with Variables
User-Defined Functions
Stored Procedures

And many More...

Working with variables

In every programming Language, variables are generally used to temporally store values in memory. T-SQL variables are created with DECLARE Command followed by variables same preceded with @ symbol and data type. By default, the values of declared variable is Null

Syntax:

```
DECLARE @Variable_Name datatype(size);  
DECLARE @Name VARCHAR(30);  
DECLARE @Name VARCHAR(50), @age INT;
```

Assign a Value to a Venable variable

Both the SET and the SELECT Command can assign the value to a variable. SET can only set the value of one variable at a time. SELECT Command retrieve data from tables and assign multiple variables values with a single Statement.

Syntax:

```
SET @Variable_Name= value;  
SELECT @Variable_Name = value, @Variable_Name=value;
```

Example:

```
SET @Salary=30000;  
SELECT @name= 'Anwaar' ; @age = 25;
```

Incrementing Variable

With the increment variable feature, we can Perform mathematical operations (like, addition, multiplication) Subtraction, and me on the variable.

Syntax:

```
SET @number(1)=10;  
SET @number = @number(+,-,*)p;
```

Example:

```
DECLAR @number INT= 100;  
SET @number += 50;  
SELECT @number;  
SET @number -=50;  
SELECT @number;  
SET @number=55  
SELECT @number;  
DECLARE @name VARCHAR(50), @age INT;  
SELECT @name = 'Anwaar' AS 'Name', @age = 25 AS 'Age';  
NAME SELECT @name, @age;  
SELECT * FROM employee;  
DECLARE @name VARCHAR(S), Salary DECIMAL (7,2);  
SELECT @name=Emp_name, @salary=Emp_Salary From  
employees;  
SELECT @name as 'Name', @salary as 'Salary';
```

Practical:

```
USE COMPANY_DB;  
DECLARE @name varchar(50);  
SFT @name= 'Anwaar';  
SELECT @name;  
SELECT @name = 'Academy';  
SELECT @name;
```

T-SQL

LESSON: 34

Script

It is a text file containing a sequence of SQL commands (statements) that can be executed together as a batch. It typically includes CREATE, INSERT, UPDATE, DELETE, SELECT, and sometimes procedural logic like conditions or loops. Scripts help automate database tasks, replicate schema/data, or run multiple operations at once.

Practical:

-- Step 1: Create table

```
CREATE TABLE students (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    grade CHAR(2)  
);
```

-- Step 2: Insert data

```
INSERT INTO students (id, name, grade) VALUES  
(1, 'Ayesha', 'A'),  
(2, 'Bilal', 'B'),  
(3, 'Danish', 'C');
```

-- Step 3: Query data

```
SELECT * FROM students;
```

T-SQL

LESSON: 35

BATCH

- A batch of SQL statements is a Group of two or more SQL Statements or a Single SQL Statement
- A Batch of SQL Statement can have:
 - I. Data Definition Language (DDL) statement
 - II. Data Manipulation Language (DAL) Statement
 - III. Data Control Language (DCL) statement

Types

1. Explicit Batch

- An Explicit Batch is two or more SQL Statements separated by Semicolons (;).

For Examples

```
Insert into employee (Emp_name, Emp_Salary)  
Values ('Brad, 4500);  
Insert into employee (Emp_name, Emp_Salary)  
values (Joe, 3600);
```

2. Procedure

- If a Procedure Contains more than one SQL Statement, it is considered to be a batch

T-SQL

LESSON: 36

Go Command

Go is not a T-SQL statement, it is a command recognized by SQL Server Utilities. Go Can be executed by any user. It requires no Permissions. It Signals the end of a batch to the SQL Server utilities.

Syntax:

Go [Count]

Where, count is a positive integer. The batch with executive the Specified number of times.

Example:

```
USE COMPANY_DB;
```

```
Go
```

```
DECLARE @Name VARCHAR (50);
```

```
SELECT @Name = 'Microsoft';
```

```
Go
```

Practical:

```
USE COMPANY_DB;
```

```
Go
```

```
DECLARE @Name VARCHAR (50);
```

```
SELECT @Name = 'Microsoft';
```

```
Go
```

The first batch contains only a USE COMPANY-DB statement to set the data base. The remaining statements use a Local variable All Local variable declaration must be grouped ma Single Batch.

T-SQL

LESSON: 37

Control of Flow

Transact-SQL (T-SQL) statements are executed in Sequential order (suppose, we have created thee Statement then, first statement will run followed be second, followed by third.). However, in many cases, we will want to interrupt-this normal flow. **"T-SQL has Keywords to control the order of execution"**

Control-of-flow Keywords

BEGIN...END

CONTINUE

BREAK

TRY...CATCH

RETURN

WHILE

WAITFOR

GOTO

IF...ELSE

THROW

T-SQL

LESSON: 38

BEGIN...END

The BEGIN...END Keyword are used to group multiple Lines into one Statement Block. In Addition, BEGIN...END can be nested it. Simply means that we place a BEGIN...END Statement within another BEGIN...END Statement.

Syntax:

```
BEGIN
    {SQL Statement or Statement Block}
```

```
END
```

Example:

```
BEGIN
    DECLARE @name VARCHAR(50), @salary
Integer
    @Dept_id VARCHAR(10)= 'D3';
Select @name=Emp_name, @Salary = Emp_Salary
FROM Employee
WHERE emp_dept_id = @dept_id;
SELECT @name as 'NAME', @salary as 'Salary';
    BEGIN
        PRINT 'Department ID: +@dept_id;
    END
END
```

Practical:

```
USE COMPANY_DB;
GO
SELECT * FROM employee;
BEGIN
    DECLARE @name VARCHAR(50), @salary
Integer
    @Dept_id VARCHAR(10)= 'D3';
Select @name=Emp_name, @Salary =
Emp_Salary FROM Employee
WHERE emp_dept_id = @dept_id;
SELECT @name as 'NAME', @salary as 'Salary';
    BEGIN
        PRINT 'Department ID: +@dept_id;
    END
END
GO
```

T-SQL**LESSON: 39****IF...ELSE**

The IF...ELSE Statement is a control-flow Statement that allows you to Execute or Skip a statement block Based on a specified Condition.

IF Condition**Syntax:**

```
IF Condition
    BEGIN
        {SQL Statement or statement Block}
    END
```

Note: If The Condition contain a SELECT statement, then it must be in Parentheses.

Example:

```
BEGIN
DECLARE @Salary DECIMAL;
SELECT @Salary = Avg (Emp_Salary) FROM employee
SELET @salary 'Avg Salary';
IF @salary > 28000
    BEGIN
        PRINT 'Average Salary is greater than 25000';
    END
END
```

Practical:

```
BEGIN
DECLARE @Salary DECIMAL;
SELECT @Salary = Avg
(Emp_Salary) FROM employee
SELET @salary 'Avg Salary';
IF @salary > 28000
    BEGIN
        PRINT 'Average Salary is
greater than 25000';
    END
END
```

IF-ELSE statement

Syntax:

```

IF Condition
BEGIN
    {SQL Statement or statement Block}
END
ELSE
BEGIN
    {SQL Statement or statement Block}
END

```

Example:

```

BEGIN
    DECLARE @Salary DECIMAL;
    SELECT @Salary = AVG (Emp_Salary) FROM
employers;
    SELECT @Salary As 'Avg Salary';
    If @Salary > 25000
    BEGIN
        Print 'Avg Salary in greater than 25000';
    END
ELSE
    BEGIN
        Print 'Avg Salary is less than 25000';
    END
END

```

Practical:

```

BEGIN
    DECLARE @Salary DECIMAL;
    SELECT @Salary = AVG (Emp_Salary)
FROM employers;
    SELECT @Salary As 'Avg Salary';
    If @Salary > 25000
    BEGIN
        Print 'Avg Salary in greater than
25000';
    END
ELSE
    BEGIN
        Print 'Avg Salary is less than
25000';
    END
END

```

T-SQL**LESSON: 40****WHILE**

The while loop statement is a control-flow Statement that allows you to execute a statement block repeatedly as long as a specified condition is TRUE. The Execution of statement in the WHILE Loop can be Controlled from inside the loop with the BREAK & CONTINUE Keywords.

Syntax:

```

WHILE Condition
BEGIN
    {SQL Statement or Statement Block}
END

```

Example/Practical:

```

BEGIN
WHILE (SELECT MIN(Emp_Salary)FROM employer) > 80,000
    BEGIN
        UPDATE employee SET Emp_Salary = Emp_Salary +10000
        Print Salary updated;
        IF (SELECT MIN (Emp_Salary)FROM employer)>= 80,000
            print 'min salary is greater or equal to 80,000;
        BREAK;
    END
END

```

TRY... CATCH

- TRY...CATCH Implements Error handling for T-SQL.
- It is similar to the Exception handling in the object-oriented programming Language Such as C++, Java, JavaScript, etc.
- A group of T-SQL Statement can be enclosed in a TRY block.
- If the statements b/w the TRY block complete without an Error, the statements b/w the CATCH block will not execute. However, if any statement inside the TRY block causes an exception, the control transfers to the statements in the CATCH Block.

Syntax:

BEGIN TRY

 {SQL statement or Block}

END TRY

BEGIN CATCH

 {SQL statement or Block}

END CATCH

Note". Any Group or T-SQL statements in a block enclosed in a BEGIN ENDBLOCK

To Retrieve the Information about the ERROR**I. ERROR-Message()**

Returns the Complete text of the generated error message.

II. ERROR-Number()

Returns the number of the Error

III. ERROR-LINE()

Returns the line number inside the routine that caused The Error.

IV. Error Procedure()

Returns the name of the stored procedure or trigger where the Error occurred,

V. Error State()

Returns the Error State, number.

VI. Error-SEVERITY()

Returns the severity.

Note: These functions Return NULL if they are called outside the scope of the CATCH block.

Example/ Practical:

BEGIN TRY

SELECT 100/5 AS 'Division';

END TRY

BEGIN CATCH

SELECT ERROR MESSAGE() AS 'Error messages';

END CATCH

T-SQL

LESSON: 42

WAIT FOR

WAITFOR block's the execution of a batch, Stored Procedure or transaction until either a specified time or time! Interval elapses, or a specified Statement. Modifies or Returns at least one Row.

TIME

The Period of time to wait.
time to Par

Syntax:

BEGIN

WAITFOR TIME to Execute {SQL
Statement or Block}
END;

**Where, time to executed can be specified either
in a Datetime, local V**

Example:

BEGIN

WAIT for TIME '18:00:00'

Select * From employee;

END

Practical:

SELECT GETDATE () as 'CT';

Go

BEGIN

WAIT for TIME '18:00:00'

END

DELAY

The time (up to a maximum of 24 hours) at Which
the

WAITFOR statement finishes.

Syntax:

BEGIN

WAITFOR DELAY time to pass
{Statement or Block of Statement}
END;

**Where, time to executed can be specified either
in a DELAY, local V**

Example:

BEGIN

WAT for DELAY '00:00:10'

Select * From employee;

END

Practical:

SELECT * FROM C_DB_dbo.employee;

GO

BEGIN

SELECT GETDATE() AS 'CT';

END

T-SQL

LESSON: 43

STORED PROCEDURE

A Stored Procedure is a Group of one or more T-SQL Statement. It Can be stored in the database

SP*

Accept input Parameters & return multiple caves. Contain Programming statements that perform operation in the database. Reform a status value to a calling program to indicate success or failure

Adv_Benefits*

- Reuse of code
- Easy to maintain
- Improve Performance
- Story Security
- Reduce server/client Network traffic

Types

1. system

Physically stored in the interval Resource database

2. User defined

It can be created in a user defined database or in all system database except the Rescore database.

3. Temporary

A form of user defined procedures. The temporary Procedures are like a permanent Procedure, except temporary procedures are stored in Tempdb.

STORED Procedure**

"Create, Modify, Delete, Execute, Grant Permission, Rename and many more"

T-SQL

LESSON: 44

SP-Create

- Two Ways to create a stored procedure

1. SP without Parameter (Simple SP)

2. SP with Parameter

Syntax: SSP

```
CREATE PROCEDURE Procedure-name
```

```
AS
```

```
BEGIN
```

```
    {SQL Statement or Blocks}
```

```
END
```

Example/ Practical:

Create a SP that returns all employees.

```
SELECTY FROM employee;
```

```
GO
```

```
CREATE PROCEDURE all employee As Details;
```

```
BEGIN SELECT * FROM employee;
```

```
END
```

```
GO
```

Save at Eg

Execat proc-all Employee Details;

T-SQL

LESSON: 45

STORED Procedure (Parameterized)

Syntax:

```
CREATE PROCEDURE  
Procedure_name (parameter list)  
As  
BEGIN  
    {SQL Statement or Block}  
END
```

Example:

Create a stored Procedure that returns all employees whose department Location is Mumbai.

Practical:

```
SELECT * FROM employee.e inner join department.d  
ON e.emp_dept_id = d.dept_id  
WHERE dept_location = "Mumbai"  
Create Procedure  
Proc all employee Detail  
(@location AS VARCHAR(100))  
As  
BEGIN  
Select FROM employee inner Join department.d  
ON e.emp_deptid = d.dept_id  
Where dept_location = @location  
END;
```

T-SQL

LESSON: 46

Stored Procedure (Modify / Alter)

- We Can modify a previously Created Stored Procedure.
- To prefer modification, we need to use ALTER Command.

Syntax:

```
ALTER Procedure Procedure_name  
As  
BEGIN  
    {SQL statement or Block}  
END  
Example:  
Modify an existing simple SP  
Proc_all Employees Details  
CREATE PROCEDURE  
Proc_all Employees Details  
AS  
BEGIN  
SELECT * FROM employee;  
END  
ALTER PROCEDURE Proc_all employee Details  
AS  
BEGIN  
SELECT e.emp_name, e.emp_Salary,  
d.dept_location  
FROM employee.e inner join department.d  
ON e.emp_deptid = d.dept_id;  
END
```

Practical:

```
Modify an existing parameterized stored Procedure  
prec_employee Detail location Wise  
CREATE PROCEDURE  
Proc-employee Details location Wise (@Location AS  
VARCHAR(100))  
AS  
BEGIN  
SELECT * FROM employee.e inner join department.d  
ON e.emp_deptid = d.dept_id;  
Where dept_location = @location  
END;  
ALTER PROCEDURE  
Proc-employee Details location Wise (@Location AS  
VARCHAR(100))  
AS  
BEGIN  
SELECT e.emp_name, e.emp_Salary, d.dept_location  
FROM employee.e inner join department.d  
ON e.emp_deptid = d.dept_id;  
WHERE d.dept_location = @location;  
END;
```

T-SQL

LESSON: 47

Rename Stored Procedure

To rename the existing Stored Procedure, we need to use system procedure.

Syntax:

EXEC SP-rename 'COMPANY_DB.proc_all Employee Details', proc_display Employee details;

Practical:

EXEC SP-rename 'COMPANY_DB.proc_all Employee Details', proc_display Employee details;

SELECT * FROM sys.sql_modules;

Drawbacks

Renaming a stored procedure does not change the name of the Corresponding object name (Sys.sql_modules) CatLog view

Changing the name or definition of a procedure can cause dependent objects to fail when the objects are not update to reflect the Changes that have been made to the procedures

Function

LESSON: 48

USER_Defined Functions

User-defined function are routines that accept Parameters, perform an action (Complex calculation) and return the result of that action as a value. The return Value Can either be a Single Scalar value or result set.

Why UDF?

1. **Modular Programming:** Create the function once store it in the database & call it any number of time in your Arya
2. **Faster execution:** Reduce the Compilation Cost of Transaction SQL Code
3. **Reduce network traffic:** Function Can' be invoked in the WHERE clause to reduce the number of rows sent to the client.

Types

System: SQL Server Provides many system function that can use to perform a variety of operations. They cannot be modified

Table_Valued: Table valued functions return a table data type.

UDF-Scalar (Create)

Before creating a function, things to know...

- User-defined function always return a value
- USER_Defined function have only input parameters for it.
- UDF cannot return multiple result sets.
- Error handling is restricted in a UDF. A UDF does not Support TRY_CATCH, @Error or RAISERROR
- SET Statement are not allowed in a UDF.
- UDF cannot call a stored Procedure.*
- UDE Can be nested. UDF can be nested up to 32 Level

Syntax:

```
CREATE FUNCTION function_name (Parameter datatype...)
```

```
RETURNS return_data type
```

```
[WITH< function_option> [...n]]
```

```
AS
```

```
    BEGIN
```

```
        Function Body
```

```
        RETURN scalar expression
```

```
    END;
```

Example / Practical:

Create a function to get employee Salary by Passing Employee Name.

```
CREATE FUNCTION Salary (@name as Varchar(50))
```

```
RETURNS decimal
```

```
BEGIN
```

```
    DECLARE @salary decimal;
```

```
    SELECT @Salary = Emp_Salary FROM employee
```

```
    WHERE Emp_name = @name;
```

```
    RETURN @salary;
```

```
END;
```

```
SELECT dbo.salary ('MIKE');
```

UDF (Table-Valued) (Create)

UDF-table-valued formation return a table data type. A TVF accepts zero or more parameters.

Types

1. **Inline TVF:** There is no function body (i.e. there is no need for a BEGIN-END block in an inline function). The table is the result set of a single SELECT Statement.

Syntax:

```
CREATE FUNCTION function_name (Parameter datatype...)
RETURNS return_datatype
AS
RETURNS statement
```

Example/ Practical:

```
CREATE A function to get Employee information by using employee Salary
CREATE FUNCTION get All employee (@salary decimal)
RETURNS TABLE
AS
RETURN
```

```
    SELECT * FROM employee
    WHERE Emp_Salary = @Salary;
```

...To Execute the Function:-

```
SELECT * FROM dbo.get All Employees(80000);
```

2. **Multi_Statement TVF:** IT Contain multiple SQL statement Enclosed in BEGING END block. The returns value is declared as a Table Variable. The RETURN statement is without a value and the declared table variable is returned.

Syntax:

```
CREAT Function f_name (Parameter datatype...)
RETURNS @Table_variable TABLE (Column_1 datatype, Column_2 datatype...)
AS
BEGIN
    SQL statement(s)
    RETURN
```

```
END;
```

Example/ Practical:

Create a function to get list of employees by Passing department.

```
CREATE FUNCTION get all Employee(@id varchar(50))
RETURNS @Table TARLE (ID int, NAME varchar(50), SALARY decimal, DEPTID varchar(50))
AS
BEGIN
    INSERT INTO @Table TARLE (ID int, NAME varchar(50), SALARY decimal,
    DEPTID varchar(50))
    SELECT * FROM employee
    WHERE emp_dept_id = @id;
    RETURN
```

```
END;
```

Function

LESSON: 51

UDF (Modify, Rename, Delete, View)

Practical:

-- Modify (drop & recreate)

```
DROP FUNCTION IF EXISTS dbo.CalcBonus;  
CREATE FUNCTION dbo.CalcBonus(@s DECIMAL)  
RETURNS DECIMAL  
AS  
BEGIN
```

```
    RETURN @s * 0.15;
```

```
END;
```

-- Rename (SQL Server)

```
EXEC sp_rename 'dbo.CalcBonus', 'ComputeBonus';
```

-- Delete (drop)

```
DROP FUNCTION IF EXISTS dbo.ComputeBonus;
```

-- View (list & definition)

```
SELECT ROUTINE_NAME FROM INFORMATION_SCHEMA.ROUTINES WHERE  
ROUTINE_TYPE='FUNCTION'; -- list all  
EXEC sp_helptext 'dbo.ComputeBonus'; -- view definition
```

TRIGGER

LESSON: 52

Trigger Introduction

A Trigger is type of **stored Procedure**, the automatically runs when an event occurs in the database served. Events are DML operations (INSERT, DELETE and UPDATE)

Types

DDL Trigger: DDL trigger is fired when DDL Statements like Drop Table, Create Table or Alter Table occurs. DDL Triggers are always After Triggers.

DML Trigger: We can create triggers on DML Statements (like 'INSERT, UPDATE and DELETE) and SP. DML triggers are of three types.

1. After Triggers
2. INSTEAD of triggers
3. CIR Triggers

TRIGGER

LESSON: 53

DML-T (Part-1)

A Trigger is type of **stored Procedure**, the automatically runs when an event occurs in the database served. Events are DML operations (INSERT, DELETE and UPDATE)

DML triggers are of three types.

After Triggers

These triggers are executed after the event of the insert, update, merge, or Delete statement is performed.

INSTEAD of triggers

These Triggers override the standard events of the TS. It can be used to perform error/value checking on one or more column. Perform additional actions before, insert, updating or deleting the row or rows

CIR Triggers

It can be either an after or INSTEAD of triggers. It can also be a DDL trigger.

Syntax:

```
CREATE Trigger Trigger_name
ON {table / view}
[With DML- Trigger_option]
{FOR| After| INSTEAD of}
{[Insert] [Update], [Delete]}
AS
BEGIN
    {SQL Statement or block of Statement}
END;
```

Before creating a trigger, things to Know...

Magic Tables

Inserted Table:

This table store copies of the effected rows during INSERT, and UPDATE statement during these transactions. New, rows are added to Both the inserted table & the TT

Deleted Table:

This table stores copies of the affected during DELETE & UPDATE Statements. During the execution of these statement, rows are deleted from the trigger table and stored into the deleted table

Example:-

Create a trigger When new employee added to employee table.

```
CREATE TRIGGER tre_message
ON employee
AFTER INSERT
AS
BEGIN
    PRINT 'New employee added to the company table'
END;
INSERT INTO employee
VALUES (2007, 'Rahul', 5000, 'D3');
```

TRIGGER

LESSON: 54

DML-T (Part-2)

Practical

```
SELECT * FROM employee_info;
CREATE TABLE employee_logs(
id int, name varchar(255),
Salary decimal,
dept_id varchar(255),
modified by varchar(255)
modified ON date);
SELECT FROM employee_logs,
CREATE Trigger tr_employee_after_inser
ON employee
After INSERT
AS
BEGIN
    INSERT INTO employee_logs SELECT emp.ID, Emp_name, Emp_Salary,
    emp_dept_id, 'Anwaar', get_delete
    From inserted
END;
CREATE & Table employee_logs;
insert into employee values (1008, 'mike', 45000 'd2)
```

--- Create Trigger for DELETE

```
CREATE TRIGGER tr_employee_after_delete
ON employee
AFTER DELETE
AS
BEGIN
    INSERT INTO employee_logs
    SELECT
        Emp_ID, Emp_name, Emp_Salary, emp_dept_id, 'Anwaar', GETDATE()
    FROM deleted;
END;
GO
```

-- Create employee_logs Table

```
CREATE TABLE employee_logs
(
    Emp_ID INT,
    Emp_name VARCHAR(50),
    Emp_Salary INT,
    emp_dept_id VARCHAR(10),
    deleted by VARCHAR(50),
    deleted date DATETIME
);
GO
```

-- Delete Record from employee table

```
DELETE FROM employee
WHERE Emp_ID = 1008;
```

TRIGGER

LESSON: 55

DML-T (Part-3)

Practical:

-- Create Employee Logs Table

```
CREATE TABLE Employee_logs (  
Emp_ID INT, Emp_name VARCHAR(50), Emp_Salary INT, Emp_dept_id VARCHAR(10), Action_Type  
VARCHAR(20), Action_Date DATETIME  
);  
GO
```

-- CREATE OR ALTER Trigger for INSERT, UPDATE, DELETE

```
CREATE OR ALTER TRIGGER Employee_all_DMLOP  
ON Employee  
AFTER INSERT, UPDATE, DELETE  
AS  
BEGIN
```

-- INSERT CONDITION

```
IF EXISTS (SELECT * FROM inserted)  
AND NOT EXISTS (SELECT * FROM deleted)  
BEGIN  
INSERT INTO Employee_logs  
SELECT  
Emp_ID, Emp_name, Emp_Salary, Emp_dept_id, 'INSERT',  
GETDATE()  
FROM inserted;  
PRINT 'Record Inserted';  
END
```

-- UPDATE CONDITION

```
ELSE IF EXISTS (SELECT * FROM inserted)  
AND EXISTS (SELECT * FROM deleted)  
BEGIN  
INSERT INTO Employee_logs  
SELECT  
Emp_ID, Emp_name, Emp_Salary, Emp_dept_id,  
'UPDATE',  
GETDATE()  
FROM inserted;  
PRINT 'Record Updated';  
END
```

-- DELETE CONDITION

```
ELSE IF EXISTS (SELECT * FROM deleted)  
AND NOT EXISTS (SELECT * FROM inserted)  
BEGIN  
INSERT INTO Employee_logs  
SELECT  
Emp_ID, Emp_name, Emp_Salary, Emp_dept_id,  
'DELETE',  
GETDATE()  
FROM deleted;  
PRINT 'Record Deleted';
```

```
END
-- NOTHING CHANGED
ELSE
BEGIN
    PRINT 'Nothing Changed!';
    RETURN;
END
END;
GO
-- Insert Record
INSERT INTO Employee
VALUES (2009, 'Vijay', 50000, 'D1');
-- Update Record
UPDATE Employee
SET Emp_Salary = 60000
WHERE Emp_ID = 2009;
-- Delete Record
DELETE FROM Employee
WHERE Emp_ID = 2009;
-- View Tables
SELECT * FROM Employee;
SELECT * FROM Employee_logs;
```

TRIGGER

LESSON: 56

DDL-T- (Part 1)

DDL trigger's fire in response to a different DDL events Correspond to SQL statements such as CREATE, ALTER, DROP, GRANT, REVOKE etc. Also, Some System Stored Procedures that Perform DDL-operations (for example, SP-rename) can also fire DDL trigger

DDL evens Reference: <https://bit.ly/1EHag> . DEET DDL events in SQL server.

Why DD2 Trigger?

- Prevent certain changes to your database Schema
- Have something occur in the Database in response to Change in your data base schema.
- Record changes or events in the database Schema

Syntax:

```
CREATE TRIGGER Trigger_name
ON {ALL SERVER| DATABASE}
[WITH DDL trigger option]
{FOR| ALTER } {Event-type-1, event-type2...}
AS
BEGIN
    {SQL statement or Blacks}
END;
```

Example/ Practical

```
CREATE a trigger when table is created
CREATE TRIGGER trOnTableCreate
ON DATABASE
FOR CREATE_TABLE
AS
BEGIN
    PRINT 'New table is created successfully';
END;
GO
-- Example Table Creation
CREATE TABLE Simple Table
(
    Id INT,
    Name VARCHAR(50)
);
GO
```

TRIGGER

LESSON: 57

DDL-T-(part-2)

EVENT DATA

- It is a built-in function
- It returns the information about the events executed the DDL trigger.
- The information is in XML format.

▮ **Event Type** (Create Table, Alter Table, Drop Table, etc...)
▮ **Post Time** (Event trigger time)
▮ **SPID** (SQL Server session ID)
▮ **Server Name** (SQL Server instance name)
▮ **Login Name** (SQL Server Login name)
▮ **UserName** (username for login, by default dbo schema as username)
▮ **Database_name** (name of database where DDL Trigger was executed)
▮ **Schema Name** (schema name of the table)
▮ **Object Name** (Name of the table)
▮ **Object Type** (Object types, such as Table, view, procedure, etc...)
▮ **TSQL Command** (Schema deployment Query which is executed by user)
▮ **Set Options** (SET Option which are applied while Creating table or Modify it)
▮ **Command Text** (Create, Alter or Drop object command)

Syntax:

```
CREATE TRIGGER Trigger_name
ON {ALL SERVER| DATABASE}
[WITH DDL trigger option]
{FOR| ALTER } {Event-type-1, event-type2...}
AS
BEGIN
    {SQL statement or Blacks}
END;
```

Practical:

Practical Example of DDL Trigger in SQL Server CREATE, RENAME, and DROP TABLE Events

-- Step 1: Create Log Table

```
CREATE TABLE DDL_Logs
(Id INT IDENTITY(1,1) PRIMARY KEY, Event Data XML, Performed By SYSNAME,
Event Type VARCHAR(200)
);
GO
```

Step 2: Create DDL Trigger

```
CREATE OR ALTER TRIGGER tr_DDLEventTrigger
ON DATABASE
FOR CREATE_TABLE, RENAME, DROP_TABLE
AS
BEGIN
    INSERT INTO DDL_Logs
```

```
(Event Data, Performed By, Event Type)
VALUES (EVENTDATA(), USER, EVENTDATA().value ('(/EVENT_INSTANCE/Event Type)[1]',
'VARCHAR(200)'));
```

```
END;
```

```
GO
```

Step 3: Create a Table

```
CREATE TABLE Demo
```

```
(Id INT, Value VARCHAR(100));
```

```
GO
```

Step 4: Rename Table

```
EXEC sp_rename 'Demo', 'Bigger Demo';
```

```
GO
```

Step 5: View Logs

```
SELECT * FROM DDL_Logs;
```

```
GO
```

Step 6: Drop Table

```
DROP TABLE Bigger Demo;
```

```
GO
```

Step 7: Again View Logs

```
SELECT * FROM DDL_Logs;
```

```
GO
```

T-SQL

LESSON: 58

Merge

Merge is A Logical combination of an insert and an update. It combines the sequence of Conditional INSERT, UPDATE and DELETE statements is a single Statement. Using Merge Statement, you can sync two different tables so that the content of the target table is modified based on difference found in the source table.

Why Merge?

- It is specially used to maintain a history of data in data warehousing during the ETL (Extract, Transform, and load) cycle.

Scenario: Suppose, tables need to be Refreshed Periodically with new data arriving from online tens action Processing (OLTP) systems. This new data may Contain Changes to existing rows in tables and/or new rows that need to be inserted.

Actions/Conditions

Rows in the Sources table are not found in the target table. Then, rows from the Source will be added to the target table. Rows in the target table are not found in the Sources table. Then, delete rows from the target table. Rows in the Source and target table have the Same Keys but, they have different non-Key Columns, then update the rows in target table with data from the Source table.

Syntax:

```
MERGE Target Table
```

```
USING Source Table
```

ON join-Conditions
 WHEN Matched
 THEN DML Statement
 WHEN NOT MATCHED BY TARGET.
 THEN DML Statement
 WHEN NOT MATCHED BY SOURCE T
 THEN DML Statement;

Note: A MUST BE TERMINATED BY A SEMICOION,

States

1. **MATCHED:** These are the rows that match the merge Condition for the matching rows, you need to update the rows Columns in the target table with values from the Source table
2. **NOT MATCHED BY TARGET:** They are the rows from the source table that does not have any matching rows in the target table. In this Cole, You need to add the rows from the source table to the target table.
3. **NOT MATCHED BY SOURCE:** These are the Rows in the target Table that does not match any rows in the source table, & you want to synchronize the target table with the data from the Source table, then you will need to use then match condition to delete nears from the target table

Syntax:

THEN Update;
 THEN insert;
 THEN THEN Delete;

Practical Example :

MERGE Statement in SQL Server

-- Step 1 : Create Database

CREATE DATABASE Airlines;
 GO

-- Step 2 : Use Database

USE Airlines;
 GO

--Step 3 : Create Flight Passengers Table

CREATE TABLE Flight Passengers
 (FlightID INT IDENTITY(1,1) PRIMARY KEY,
 FirstName VARCHAR(80) NOT NULL,
 FlightCode VARCHAR(20) NOT NULL,
 FlightDate DATE NOT NULL,
 Seat VARCHAR(8)

);
 GO

--Step 4 : Insert Data into Flight Passengers

INSERT INTO Flight Passengers
 (FirstName, FlightCode, FlightDate, Seat)
 VALUES ('SMITH', 'SQL244', GETDATE(), '17F'), ('ADAM', 'SQL204', GETDATE(), '20A'),
 ('MIKE', 'SQL111', GETDATE(), '4B');
 GO

--Step 5 : View Passenger Table

```
SELECT * FROM Flight Passengers;  
GO
```

--Step 6 : Create CheckIN Table

```
CREATE TABLE CheckIN  
(FirstName VARCHAR(80), FlightCode VARCHAR(20), Seat VARCHAR(8));  
GO
```

-- Step 7 : Insert Data into CheckIN Table

```
INSERT INTO CheckIN  
VALUES  
( 'SMITH', 'SQL244', '18A'),  
( 'JOHN', 'SQL500', '10C');  
GO
```

--Step 8 : View CheckIN Table

```
SELECT * FROM CheckIN;  
GO
```

--Step 9 : MERGE Statement

```
MERGE Flight Passengers AS F  
USING CheckIN AS C  
ON C.FirstName = F.FirstName  
AND C.FlightCode = F.FlightCode  
WHEN MATCHED  
THEN  
    UPDATE  
    SET F.Seat = C.Seat  
WHEN NOT MATCHED BY TARGET  
THEN  
    INSERT  
    (FirstName, FlightCode, FlightDate, Seat)  
    VALUES  
    ( C.FirstName, C.FlightCode, GETDATE(), C.Seat)  
WHEN NOT MATCHED BY SOURCE  
THEN  
    DELETE;  
GO
```

--Step 10 : Final Output

```
SELECT * FROM Flight Passengers;  
GO
```

INDEX

In general, index is used to measure the Performance. Database Systems uses indices to provide fast access to relational data. It is a special type of physical data Structure used to access one or more data rows fast. Database index Can Change each time the Corresponding data is changed.

Differences of Index

1. Book Index

- A book reader can decide whether or not to use the book's index.
- A particular book's index is edited together with the book and does not change at all. This means that you can find a topic exactly on the page where it is determined in the index.

2. Database Index

- The system component called the query optimizer decides whether or not to use an existing index.
- A database index can change each time the corresponding data is changed.

Example:

Find employee names whose salary is greater than 50000

```
SELECT * FROM Employee  
WHERE SALARY > 50000;  
CREATE INDEX IDX_EMPLOYEE_SALARY  
ON EMPLOYEE (SALARY ASC);
```

SYNTAX:

```
CREATE INDEX index_name ON table name;
```

Practical:

```
CREATE TABLE employee  
( SELECT * FROM Employer  
  WHERE Salary > 50000;  
  CREATE INDEX idx_employee_Salary  
  ON employee (SALARY ASC);  
  NAME VARCHAR(50),  
  SALARY DECIMAL,  
  LOCATION VARCHAR(50)  
);  
INSERT INTO employee  
VALUES  
(1003, 'SMITH', 65000, 'USA'),  
(1001, 'JAMES', 30000, 'INDIA'),  
(1002, 'MIKE', 48000, 'INDIA'),  
(1004, 'JOHN', 55000, 'USA');  
SELECT * FROM employee;
```

T-SQL

LESSON: 60

CLUSTERED INDEX

A clustered index determines the physical order of the data in a table. Hence, a table can have only one clustered index. When a clustered index is created, the database engine sorts the data in the table based on the defined index key(s) and stores the table in that order. Example, a telephone book. A telephone book is always in sorted order, based on the last name of the individual followed by the first name. The sorted order makes it easy to find the phone number of the person you are looking for.

Things to remember

A Primary Key Constraint creates clustered index automatically if there is no clustered Index exists off the table. An index can contain multiple columns, knows Composite index.

Syntax:

```
CREATE CLUSTERED INDEX index-name  
ON table-name(Column_Name <ASCII DESC7);
```

Practical:

```
SELECT * FROM Employee;  
CREATE CLUSTERED INDEX index-employee-name  
ON employee (name ASC);
```

T-SQL

LESSON: 61

NONCLUSTERED INDEX

- A nonclustered index does not change the physical order of the rows in the table.
- In other (simple) words, a nonclustered index is similar to an index in a textbook. The data is stored in one place, the index in another place. The index will have pointer to the storage location of the data.
- Since, the index is stored separately from the actual data, a table can have more than one nonclustered index.

Syntax:

```
CREATE NONCLUSTERED INDEX index-name  
ON table-name(Column_Name (ASC | DESC));
```

Example:-

```
CREATE NONCLUSTERED INDEX index_employee_name  
ON employee (name ASC);
```

Note: It is faster in searching for the values That are not in the range.

Practical:

```
SELECT * FROM Employee;  
Create NONCLUSTERED INDEX index_employee_name  
ON Employee (NAME ASC);  
Select * from employee where name = "mike"
```

UNIQUE INDEX

A unique index ensures that the index key contains no duplicate values. There are no differences b/w creating a Unique Constraint and creating a unique index. Creating a UNIQUE Constraint on the column makes the purpose of the index Clear.

Implementation:

- A. **Primary KEY:** When you create a primary key constraint, a unique clustered index on the column or Columns is automatically.
- B. **unique constraint:** unique constraint, a When you create a unique non clustered index is created to enforce a unique constraint by default
- C. **Index Independent of a constraint:** Multiple unique Nonclustered indexes can be defined on a table.

Syntax:

```
CREATE UNIQUE NONCLUSTERED INDEX index_name  
ON table_name (Column_Name);
```

Example:

```
CREATE UNIQUE NONCLUSTERED INDEX idx_employee_id  
ON employee(id);
```

Practical:

```
CREATE TABLE department  
(did int primary key, dname varchar(50), dloc varchar(50))  
INSERT INTO department VALUES (2, 'SALES', 'MUMBAI'),  
(1, 'ACCOUNTS', 'PUNE');  
SELECT * FROM department;  
SELECT * FROM department;  
ALTER TABLE department  
ADD CONSTRAINT UQ_department_did  
UNIQUE CLUSTERED (did);  
CREATE UNIQUE NONCLUSTERED INDEX idx_department_dloc  
ON department (dloc)
```

View

A view does not require any storage in a data base. It is a Saved query. It acts as a filter on the tables referenced in the view query. The main use care of a view to maintain the Security at row and Column Level.

Syntax:

```
CREATE VIEW View_name  
AS [simple select query] [Join query];
```

Example:

```
CREATE VIEW V_Employee Details  
AS SELECT empName, Emp_Salary, loc FROM employee;
```

Practical:**-- View 1 : Display Employee Name and Salary**

```
CREATE VIEW V_EmpNameSalary  
AS  
SELECT Name, Salary  
FROM Employee;  
GO
```

-- View Output

```
SELECT * FROM V_EmpNameSalary;  
GO
```

View 2 : Display Employee Details with Department Name

```
CREATE VIEW V_EmpDetailsWithDeptName  
AS  
SELECT  
    E.Name,  
    E.Salary,  
    E.Location,  
    D.DName AS DepartmentName  
FROM Employee E  
INNER JOIN Department D  
ON E.DeptID = D.DID;  
GO
```

-- View Output

```
SELECT * FROM V_EmpDetailsWithDeptName;  
GO
```

View 3 : Display Employee Details without Salary

```
CREATE VIEW V_EmployeePublicData  
AS  
SELECT  
    E.Name,  
    E.Location,  
    D.DName AS DepartmentName  
FROM Employee E  
INNER JOIN Department D  
ON E.DeptID = D.DID;  
GO
```

-- View Output

```
SELECT * FROM V_EmployeePublicData;  
GO
```

View 4 : Display Employees who work in India

```
CREATE VIEW V_EmpWorkForIndia  
AS  
SELECT *  
FROM Employee  
WHERE Location = 'India';  
GO
```

-- View Output

```
SELECT * FROM V_EmpWorkForIndia;  
GO
```

-- View Definition

```
SP_HELPTEXT 'V_EmpWorkForIndia';  
GO
```

T-SQL

LESSON: 64

CTE (Common Table Expression)

CTE is a Temporary named result set. A CTE must be followed by a single SELECT, INSERT, UPDATE or DELETE. Statement that references some or all the CTE Columns, A CTE can be specified a CREATE view Statement as part of the defining SELECT statement of the Multiple CTE query definitions can be defined view.

Syntax:

```
With cte_name AS(cte_query) followed query  
With cte_name (col1, col2...) AS (cte_query)  
Followed_query
```

Example:

```
With cte_avg_salary AS  
(SELECT Avg (Salary) as Avg_Salary From Employee)  
SELECT Avg_Salary From cte_avg_salary;
```

Practical:

-- Step 1 : Create Employee Table

```
CREATE TABLE Employee  
(EmpID INT PRIMARY KEY, Name VARCHAR(50), Salary DECIMAL(10,2), DeptID INT);  
GO
```

-- Step 2 : Insert Data into Employee Table

```
INSERT INTO Employee  
VALUES(1, 'SMITH', 50000, 101), (2, 'JOHN', 45000, 102), (3, 'ADAM', 60000, 101), (4, 'MIKE', 55000, 103), (5, 'JAMES',  
70000, 102);  
GO
```

--Step 3 : Create Department Table

```
CREATE TABLE Department
```

```
( DID INT PRIMARY KEY, DName VARCHAR(50));  
GO
```

--Step 4 : Insert Data into Department Table

```
INSERT INTO Department  
VALUES (101, 'HR'), (102, 'IT'), (103, 'SALES');  
GO
```

--Step 5 : View Tables

```
SELECT * FROM Employee;  
SELECT * FROM Department;  
GO
```

Example 1 : Average Salary using CTE

```
WITH CTE_AvgSalary AS  
(SELECT AVG(Salary) AS Avg_Salary FROM Employee)  
SELECT Avg_Salary FROM CTE_AvgSalary;  
GO
```

Example 2 : Employee Count Department Wise

```
WITH CTE_EmpCount  
(DeptID, EmployeeCount)  
AS  
( SELECT  
    DeptID,  
    COUNT(*) AS EmployeeCount  
  FROM Employee  
  GROUP BY DeptID  
)  
SELECT  
    D.DName, C.EmployeeCount  
  FROM Department D  
  INNER JOIN CTE_EmpCount C  
    ON D.DID = C.DeptID;  
GO
```

Transaction Control Language

If a transaction is successful, all of the data modifications made during the transaction are Committed and become a permanent part of the data base. If a transaction encounters errors and must be canceled or roll Back, then all of the data modifications are erased.

COMMIT

Roll Back

Save

Syntax:

(EGIN TRANSACTION;

SQL Statement

Commit | Rollback | Save)

Practical:

BEGIN TRANSACTION;

-- Start transaction (all operations will be treated as one unit)

UPDATE Employee

SET Salary = Salary + 5000

WHERE EmpID = 1;

-- Increase salary of employee with ID 1

SAVE TRANSACTION SP1;

-- Create savepoint (checkpoint) inside transaction

DELETE FROM Employee

WHERE EmpID = 2;

-- Delete employee with ID 2

ROLLBACK TRANSACTION SP1;

-- Undo all changes after savepoint SP1 (deletes undone, update remains)

COMMIT;

-- Permanently save the remaining changes to database